Poisson and negative-binomial regression

Inference labs use the five-step template: Hypothesis → Visualise → Assumptions → Conduct → Conclude.

Learning objectives

- Fit a Poisson GLM for counts with `glm(family = poisson)` and use an offset for exposure.
- Diagnose overdispersion and switch to negative-binomial regression with `MASS::glm.nb` when needed.
- Report incidence rate ratios with intervals.

Prerequisites

Logistic regression.

Background

Counts with a known exposure are the natural habitat of the Poisson GLM: number of events per person-year, per square metre, per trap night. The canonical link is the log, so the exponentiated coefficient is an incidence rate ratio. Exposure enters the model as an offset — a term with a fixed coefficient of 1 on the log scale.

The Poisson model assumes variance equals the mean. Most real count data violate this, with variance growing faster than the mean. Overdispersion inflates Type-I error when ignored. Checking the ratio of residual deviance to its degrees of freedom gives a quick read; when it is substantially above 1, the negative-binomial GLM (or a quasi-Poisson variance) is a better fit.

Poisson regression without an offset is a common trap. If the denominators vary, a coefficient on an exposure variable is confounded with exposure; the offset separates the two and restores the interpretation of the log-rate ratio.

Setup

```
library(tidyverse)
library(broom)
library(MASS)
set.seed(42)
theme_set(theme_minimal(base_size = 12))
```

1. Hypothesis

Simulate counts of events with a dispersion parameter larger than 1 (so negative-binomial is the correct model), plus a known exposure.

2. Visualise

```
x <- rnorm(n)
exposure <- runif(n, 0.5, 2)
mu <- exposure * exp(0.5 + 0.4 * x)
# negative-binomial counts with theta = 2 (overdispersion)
y <- rnegbin(n, mu = mu, theta = 2)
```

```r
dat <- tibble(x, exposure, y)

ggplot(dat, aes(x, y)) +
  geom_point(alpha = 0.6) +
  labs(x = "x", y = "Count")
```

3. Assumptions

Independent counts; correct mean structure; for Poisson, variance = mean.

```r
# dispersion
dev_over_df <- deviance(fit_p) / df.residual(fit_p)
dev_over_df
```

Dispersion » 1 signals negative-binomial.

4. Conduct

```r
bind_rows(
  tidy(fit_p, conf.int = TRUE, exponentiate = TRUE) |> mutate(model = "Poisson"),
  tidy(fit_nb, conf.int = TRUE, exponentiate = TRUE) |> mutate(model = "NB")
) |>
  filter(term == "x") |>
  dplyr::select(model, estimate, conf.low, conf.high, std.error)
```

The estimates are similar but the NB standard error is larger and honest.

5. Concluding statement

> Event counts increased with x (NB IRR per unit x = `round(exp(coef(fit_nb))["x"], 2)`; 95% CI `round(exp(confint(fit_nb))["x", 1], 2)` to `round(exp(confint(fit_nb))["x", 2], 2)`). Poisson residual deviance indicated overdispersion; we therefore report the negative-binomial fit as primary.

Common pitfalls

- Forgetting the offset when denominators vary.
- Using Poisson when overdispersion is present.
- Reporting the rate ratio without an interval.

Further reading

- McCullagh P, Nelder JA. *Generalized Linear Models*, ch. 6.
- Hilbe JM. *Negative Binomial Regression*.
- Zeileis A, Kleiber C, Jackman S (2008), *Regression models for count data*.

Session info

See also — chapter index

- Methods in this chapter
- Find your method (Chapter 0)